Software Evolution and Maintenance

A Practitioner's Approach

Chapter 2

Taxonomy of Software Maintenance and Evolution

© Tripathy & Naik

Outline of the Chapter

2.1 Classification of Software Maintenance Activities

2.2 Categories of Maintenance Concepts

2.3 Evolution of Software Systems

2.4 Maintenance of COTS-based Systems

© Tripathy & Naik

Software Maintenance

- In circa 1972, R. G. Canning in his landmark article

  "The Maintenance 'Iceberg',"

  discussed the problems of software maintenance.
- Practitioners took a narrow view of maintenance as
  - correcting errors, and
  - enhancing the functionalities of the software.
- The ISO/IEC 14764 standard defines software maintenance as
  - "... the totality of activities required to provide cost-effective support to a software system. Activities are performed during the pre-delivery stage as well as the post-delivery stage."
    - Post-delivery activities includes changing software, providing training, and operating a help desk.
    - Pre-delivery activities include planning for post-delivery operations.

© Tripathy & Naik

Swanson's classification of Software Maintenance Activities

Swanson in 1976 defines three categories of software maintenance activities :

- Corrective maintenance: The purpose of corrective maintenance is to correct failures: processing failures and performance failures.

- Adaptive maintenance: The purpose of adaptive maintenance is to enable the system to adapt to changes in its data environment or processing environment.

- Perfective maintenance: The purpose of perfective maintenance is to make a variety of improvements, namely, user experience, processing efficiency, and maintainability.

Included in the Standard IEEE 1219-1992 and in the revised version of 1998.

© Tripathy & Naik

Corrective Maintenance

- Corrective maintenance: The purpose of corrective maintenance is to correct failures: processing failures and performance failures.
- Examples of corrective maintenance:
  - A program producing a wrong output is an example of processing failure.
  - Similarly, a program not being able to meet real-time requirements is an example of performance failure.
- The process of corrective maintenance includes isolation and correction of defective elements in the software.
- There is a variety of situations that can be described as corrective maintenance such as correcting a program that aborts or produces incorrect results.
- Basically, corrective maintenance is a reactive process, which means that corrective maintenance is performed after detecting defects with the system.

© Tripathy & Naik

Adaptive Maintenance

- Adaptive maintenance: The purpose of adaptive maintenance is to enable the system to adapt to changes in its data environment or processing environment.
- This process modifies the software to properly interface with a changing or changed environment.
- Adaptive maintenance includes system changes, additions, deletions, modifications, extensions, and enhancements to meet the evolving needs of the environment in which the system must operate.
- Examples of Adaptive maintenance are:
  - changing the system to support new hardware configuration;
  - converting the system from batch to on-line operation; and
  - changing the system to be compatible with other applications.

© Tripathy & Naik

Perfective Maintenance

- Perfective maintenance: The purpose of perfective maintenance is to make a variety of improvements, namely, user experience, processing efficiency, and maintainability.
- Examples of perfective maintenance are:
  - the program outputs can be made more readable for better user experience;
  - the program can be modified to make it faster, thereby increasing the processing efficiency;
  - and the program can be restructured to improve its readability, thereby increasing its maintainability.
- Activities for perfective maintenance include restructuring of the code, creating and updating documentations, and tuning the system to improve performance.
- It also includes “reengineering”.

© Tripathy & Naik

Preventive Maintenance (1/2)

Some authors later used the term Preventive Maintenance to refer to a new category of maintenance activity:

- Preventive maintenance: The purpose of preventive maintenance is to prevent problems from occurring by modifying software products.
- Basically, one should look ahead, identify future risks and unknown problems, and take actions so that those problems do not occur.
- Preventive maintenance is very often performed on safety critical and high available software systems.

Preventive Maintenance (2/2)

- The concept of "software rejuvenation" is a preventive maintenance measure to prevent, or at least postpone, the occurrences of failures (crash) due to continuously running the software system.
- It involves occasionally terminating an application or a system, cleaning its internal state, and restarting it.
- Rejuvenation may increase the downtime of the application; however, it prevents the occurrence of more severe failures.

The term preventive maintenance has been also used to refer to maintenance activities aiming at improving the structure of a system (restructuring/reengineering).

However, later it has been standardized with a meaning related to software correction.

© Tripathy & Naik

Revised Classification of Software Maintenance

Kitchenham et al. in 1999 organize maintenance activities into two categories:

- Corrections
  - Activities in this category are designed to fix defects in the system, where a defect is a discrepancy between the expected behavior and the actual behavior of the system.
- Enhancements
  - Activities in this category are designed to effect changes to the system.
  - It is further divided into three subcategories as follows:
    - enhancement activities that modify some of the existing requirements;
    - enhancement activities that add new system requirements;
    - enhancement activities that modify the implementation without changing the requirements.

This classification was refined and included in the Software Engineering Body of Knowledge (SWEBOK) and in the standard ISO-IEC & IEEE 14764-2006 (which superseded the standard IEEE 1219-1998).

© Tripathy & Naik

Standard ISO-IEC IEEE 14764-2006: Correction activities

- *Corrective Maintenance*
  - The reactive modification of a software product performed after delivery to correct discovered problems
  - *Emergency Maintenance*
    - An unscheduled modification performed to temporarily keep a system operational pending corrective maintenance
- *Preventive Maintenance*
  - The modification of a software product after delivery to detect and correct latent faults in the software product before they become operational faults

Note how preventive maintenance now takes on its definitive connotation.

About 80% of maintenance activities are non-corrective activities …

Standard ISO-IEC IEEE 14764-2006: Enhancement activities

- A modification to an existing software product to satisfy a new requirement:

- Perfective
  - Enhancements for users, improvement of program documentation, and recoding to improve software performance, maintainability, or other software attributes
- Adaptive
  - The modification of a software product, performed after delivery, to keep a software product usable in a changed or changing environment

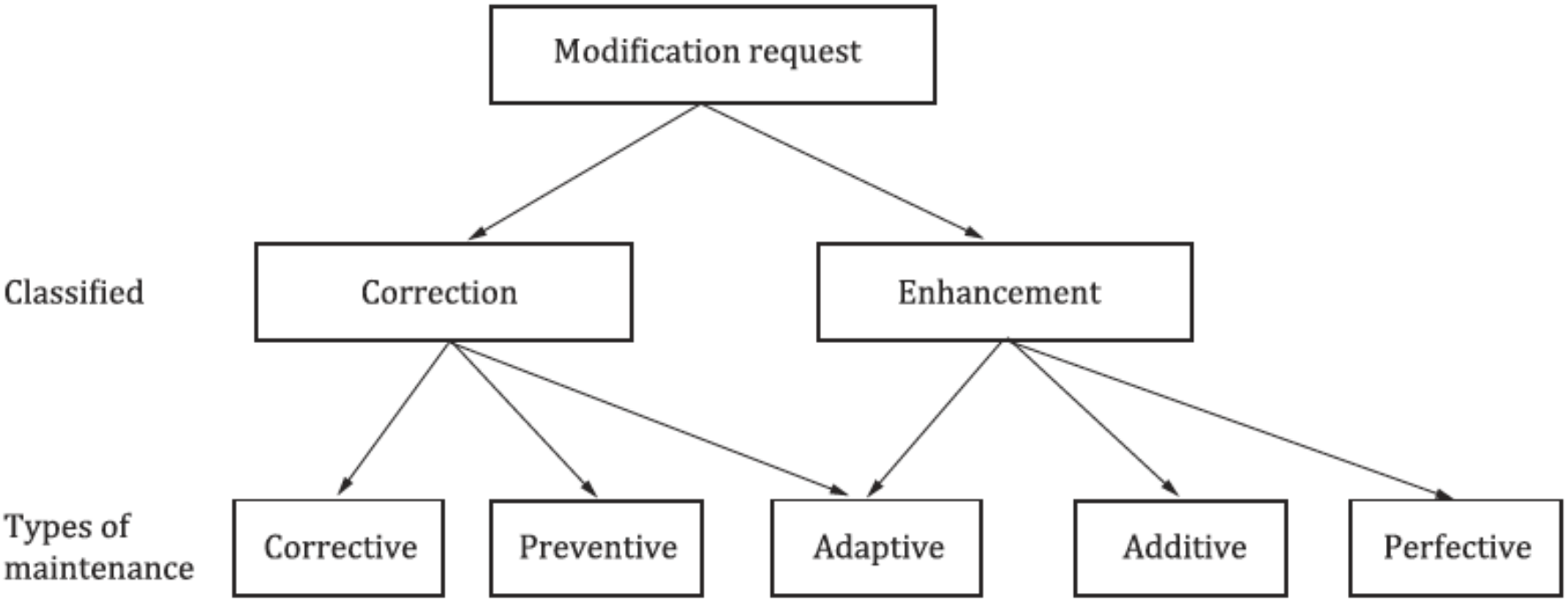
Anything missing?

- How do we classify maintenance activities adding new functionalities to the system?
- Should adaptive maintenance activities always be considered as enhancements?
  - How about the year 2000 maintenance?
  - How about the Euro conversion maintenance?

Standard ISO-IEC 14764-2022: Revising maintenance activity

- Additive maintenance
  - modification of a software product performed after delivery to add functionality or features to enhance
  - A new category of enhancement activity …
- Adaptive maintenance
  - In some organizations, adaptive maintenance is not considered to be an enhancement
    - Adaptive maintenance can be classified as enhancement, when the software is to be adapted to its evolving and changing environment or different environments on which the software is not initially intended to operate.
    - Additionally, adaptive maintenance can be classified as correction, when the software is to be adapted to its environment that has already changed

Standard ISO-IEC 14764-2022: Types of software maintenance



This is the currently accepted classification …

2.2 Categories of Maintenance Concepts

The four categories and the concepts that influence the maintenance process have been illustrated in the following Fig. 2.3.

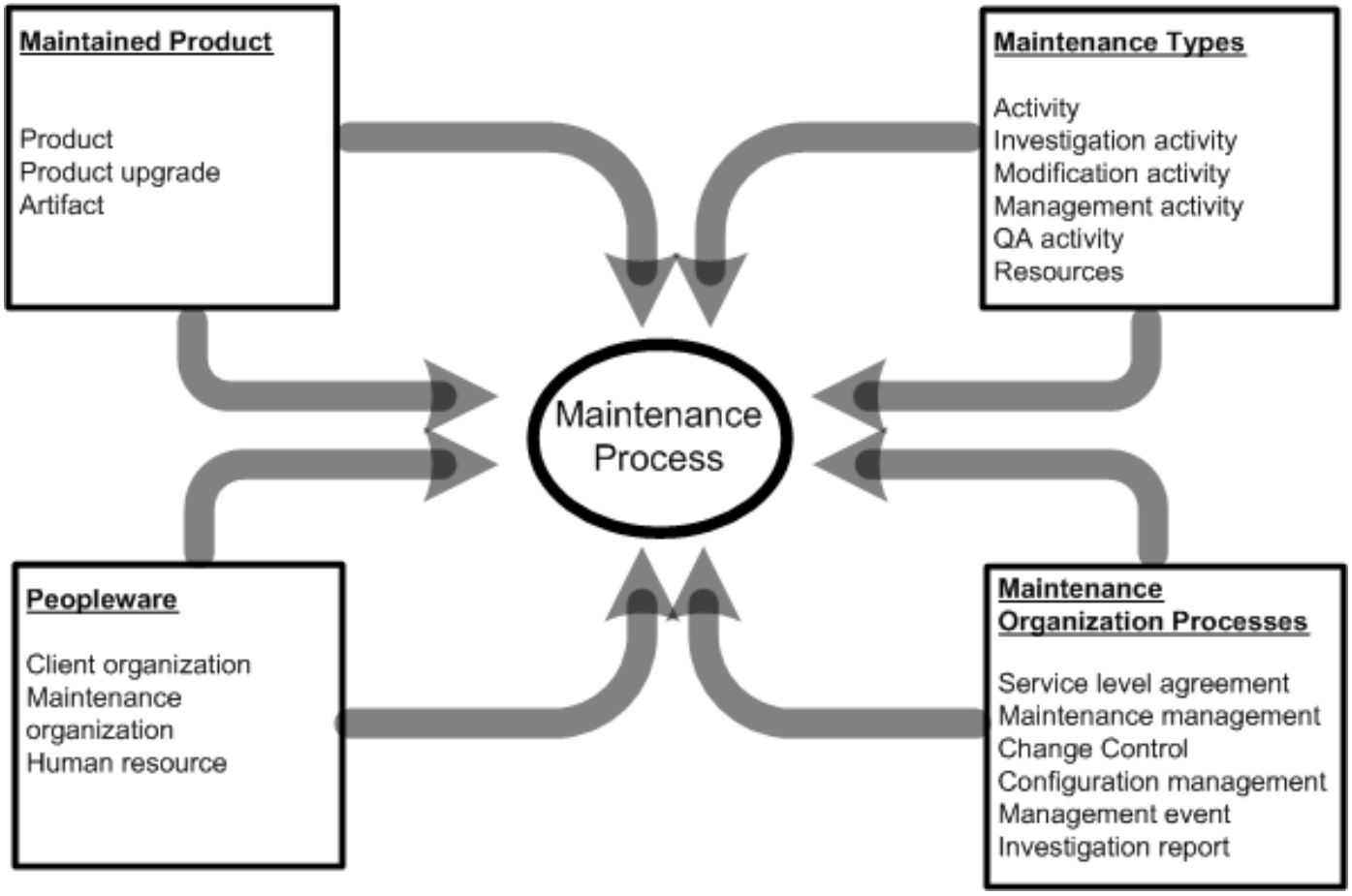


Figure 2.3 Overview of concept categories affecting software maintenance

© Tripathy & Naik

2.2.1 Maintained Product

The maintained product dimension is characterized by three concepts:

- **Product**: A product is a coherent collection of several different artifacts. Source code is the key component of a software product.

- **Product upgrade:** Baseline is an arrangement of related entities that make up a particular software configuration. Any change or upgrade made to a software product relates to a specific baseline.
An upgrade can create a new version of the system being maintained, a patch code for an object, or even a notice explaining a restriction on the use of the system.

- **Artifacts:** A number of different artifacts are used in the design of a software product. One can find the following types of artifacts: textual and graphical documents, component off-the-shelf products, and object code components.

The key elements of the maintained product are size, age, application type, composition and quality.

© Tripathy & Naik

2.2.2 Maintenance Types

Four types of maintenance activities are defined:

- **Investigation activity:** This kind of activities evaluate the impact of making a certain change to the system.
- **Modification activity:** This kind of activities change the system's implementation.
- **Management activity:** This kind of activities relate to the configuration control of the maintained system or the management of the maintenance process.
- **Quality assurance activity:** This kind of activities ensure that modifications performed on a system do not damage the integrity of the system.

Activity: An activity accepts some artifacts as inputs and produces new or changed artifacts.

Artifacts: Artifacts include plans, documents, system representations, and source and object code items. Artifacts are created during the software development process and changed during maintenance.

© Tripathy & Naik

2.2.3 Maintenance Organization Processes

Two different levels of maintenance processes are followed within a maintenance organization:

- **Individual-level maintenance processes** followed by maintenance personnel to implement a specific change request, and

- **Organization-level process** followed to manage the change requests from maintenance personnel, users, and customers/clients.

© Tripathy & Naik

2.2.3 Maintenance Organization Processes

Three major elements of a maintenance organization are

- **Event management:** The stream of events, namely, all the change requests from various sources, received by the maintenance organization is handled in an event management process.

- **Configuration management:** A system's integrity is maintained by means of a configuration management process. Integrity of a product is maintained in terms of its modification status and version number.

- **Change control system:** Evaluation of results of investigations of maintenance events is performed in a process called change control. Based on the evaluation, the organization approves a system change.

© Tripathy & Naik

2.2.3 Maintenance Organization Processes

- A maintenance organization handles maintenance change requests from:
  1. Users,
  2. Customers, and
  3. maintainers.

- After an initial investigation of a change request, a management process is put in place for approving change activities.
- Approval of a change request is normally the responsibility of a change control board.
- A proposed modification activity is scheduled only after the modification is approved by the board and a Service Level Agreement (SLA) is signed with the client.
- Service level agreement (SLA) is a contract between the customers and the providers of a maintenance service. Performance targets for the maintenance services are specified in the SLA.

2.2.3 Maintenance Organization Processes

In general, maintenance organizations use three different support levels to organize the staff:

- **Level 1:** This group files problem reports and identifies the technical support person who can best assist the person reporting a problem.

- **Level 2:** This level includes experts who know how to communicate with users and analyze their problems. These people recommend quick fixes and temporary workarounds.

- **Level 3:** This level includes programmers who can perform actual changes to the product software.

© Tripathy & Naik

2.2.4 Peopleware

- Maintenance activities cannot ignore the human element, because software production and maintenance are human intensive activities.

- The three people-centric concepts related to maintenance are as follows:

1. **Maintenance organization:** This is the organization that maintains the product(s).

2. **Client organization:** A client organization uses the maintained system.

3. **Human resource:** Human resource includes personnel from the maintenance and client organizations.

© Tripathy & Naik

2.2.4 Peopleware

Finally, the following user and customer issues affect maintenance:

- **Size:** The size of the customer base and the number of licenses they hold affect the amount of effort needed to support a system.
- **Variability:** High variability in the customer base impacts the scope of maintenance tasks.
- **Common goals:** The extent to which the users and the customer have common goals affect the SLAs.

Ultimately, customers fund maintenance activities.
If the customers do not have a good understanding of the requirements of the actual users, some SLAs may not be appropriate to the end users.

© Tripathy & Naik

2.3 Evolution of Software Systems

- ~~The term evolution was used by Mark I. Halpern in circa 1965 to define the dynamic growth of software.~~
- The term evolution was introduced by M. Lehman in the ’70s to define the dynamic growth of software.

- It attracted more attention in the 1980s after Meir M. Lehman proposed eight broad principles about how certain types of software systems evolve.

- Bennett and Rajlich researched the term “software evolution,” but found no widely accepted definition of the term.

- Some researchers and practitioners used the term **software evolution** as a substitute for the term **software maintenance**.

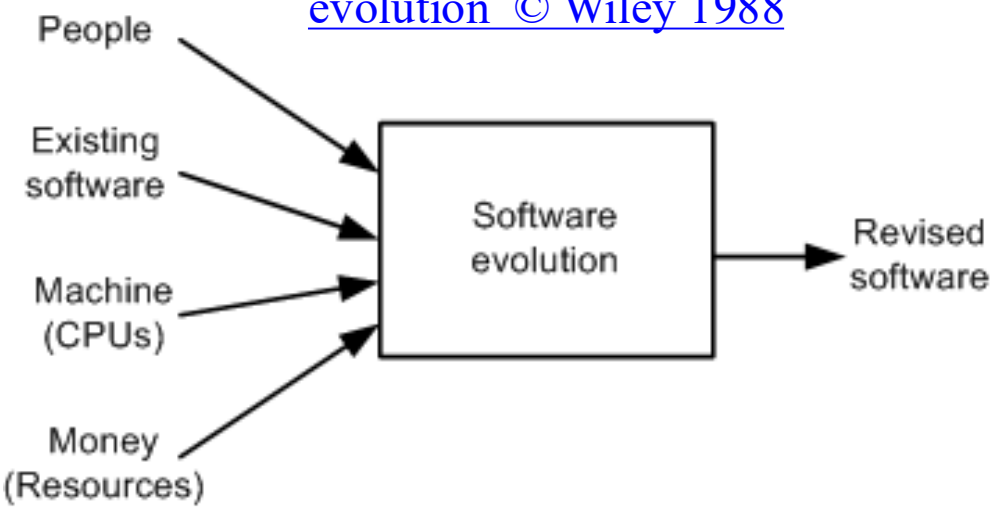
© Tripathy & Naik

2.3 Evolution of Software Systems

Lowell Jay Arthur distinguished the two terms as follows:

- Maintenance means preserving software from decline or failure.
- Evolution means a continuously changing software from a worse state to a better state.
- Software evolution is like a computer program, with inputs, processes, and outputs.
- Keith H. Bennett and Jie Xu use **maintenance** for all post-delivery support, whereas they use **evolution** to refer to perfective modifications - modifications triggered by changes in requirements.

Figure 2.4 Inputs and outputs of software evolution © Wiley 1988



© Tripathy & Naik

2.3 Evolution of Software Systems

Software evolution is studied with two broad, complementary approaches:

- **Explanatory (What/Why):**

1. This approach attempts to explain the causes of software evolution, the processes used, and the effects of software evolution.
2. The explanatory approach studies evolution from a scientific view point.

- **Process improvement (How):**

1. This approach attempts to manage the effects of software evolution by developing better methods and tools, namely, design, maintenance, refactoring, and reengineering.
2. The process improvement approach studies evolution from an engineering view point.

© Tripathy & Naik

2.3.1 SPE Taxonomy

- The abbreviation SPE refers to
  - **S (Specified),**
  - **P (Problem),** and
  - **E (Evolving)** programs

- In circa 1980, Meir M. Lehman proposed an SPE classification scheme to explain the ways in which programs vary in their evolutionary characteristics.

Lehman observed a key difference between:

- Software developed to meet a fixed set of requirements, and
- Software developed to solve a real world problem which changes with time.

- The observation leads to the identification of types **S (Specified), P (Problem),** and **E (Evolving)** programs.

© Tripathy & Naik

2.3.1 SPE Taxonomy

S-type (Specified) programs are programs whose function is formally defined by and derivable from a specification. They have the following characteristics:

- All the non-functional and functional program properties, that are important to its stakeholders, are formally and completely defined.
- Correctness of the program with respect to its formal specification is the only criterion of the acceptability of the solution to its stakeholders.

Examples of **S-type programs**:

(i) Calculation of the lowest common multiple of two integers.

(ii) Perform matrix addition, multiplication, and inversion.

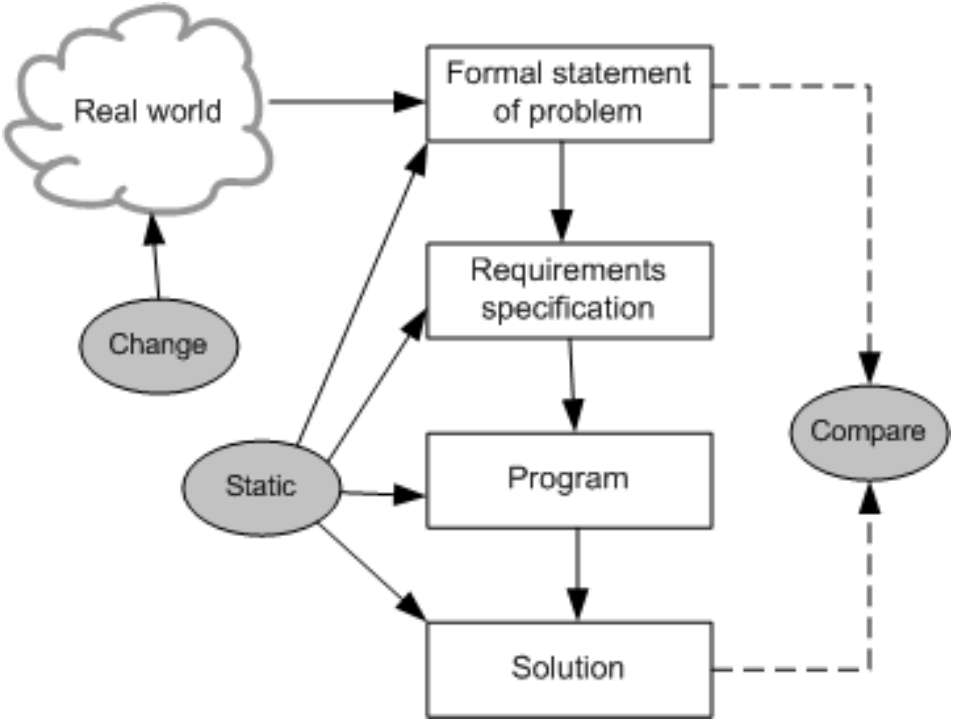


Figure 2.5 S-type programs

© Tripathy & Naik

2.3.1 SPE Taxonomy

- **P-type (Problem) program** is based on a practical abstraction of the problem, instead of relying on a completely defined specification.
- The problem can be precisely formulated but its solution must inevitably reflect an approximation of the real world
- Example: A program that play chess.
- The **P-type program** resulting from the changes cannot be considered a new solution to a new problem. Rather, it is a modification of the old solution to better fit the existing problem.
- In addition, the real world may change, hence the problem changes.

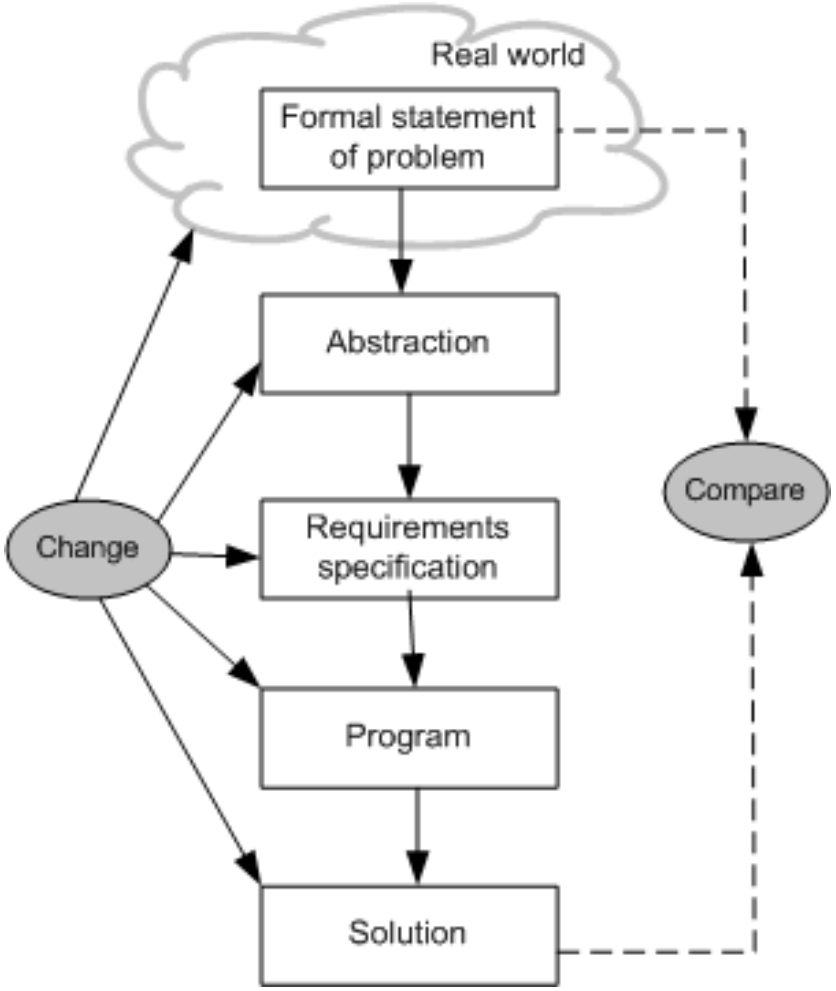


Figure 2.6 P-type programs

© Tripathy & Naik

2.3.1 SPE Taxonomy

An **E-type (Evolving) program** is one that is embedded in the real world and it changes as the world does.

- These programs mechanize a human or society activity, make simplifying assumptions, and interface with the external world by requiring or providing services.
- The acceptance of an E-type program entirely depends upon the stakeholders' opinion and judgment of the solution.

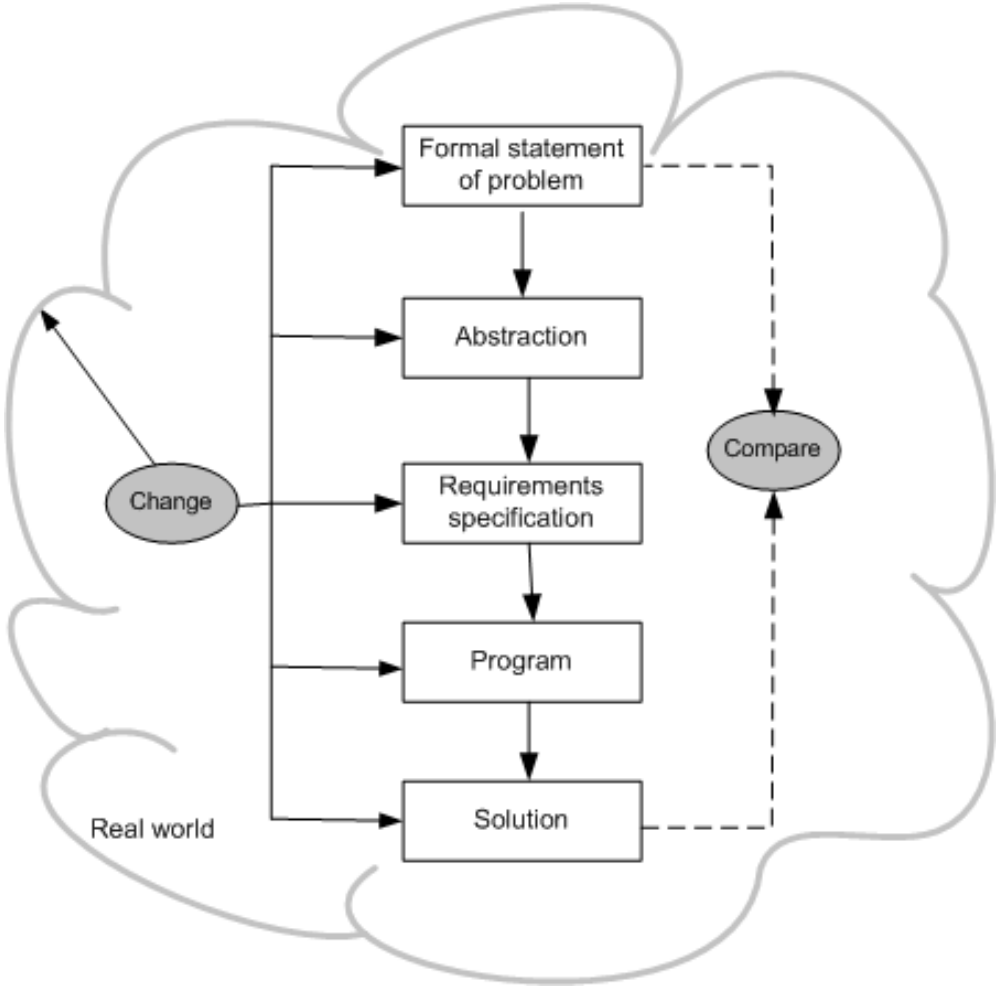


Figure 2.7 E-type programs

© Tripathy & Naik

2.3.1 SPE Taxonomy

- The first characteristic of an E-type program is that the outcome of executing the program is not definitely predictable.

- The second characteristic is that program execution changes its operational domain, and the evolution process is viewed as a feedback system.

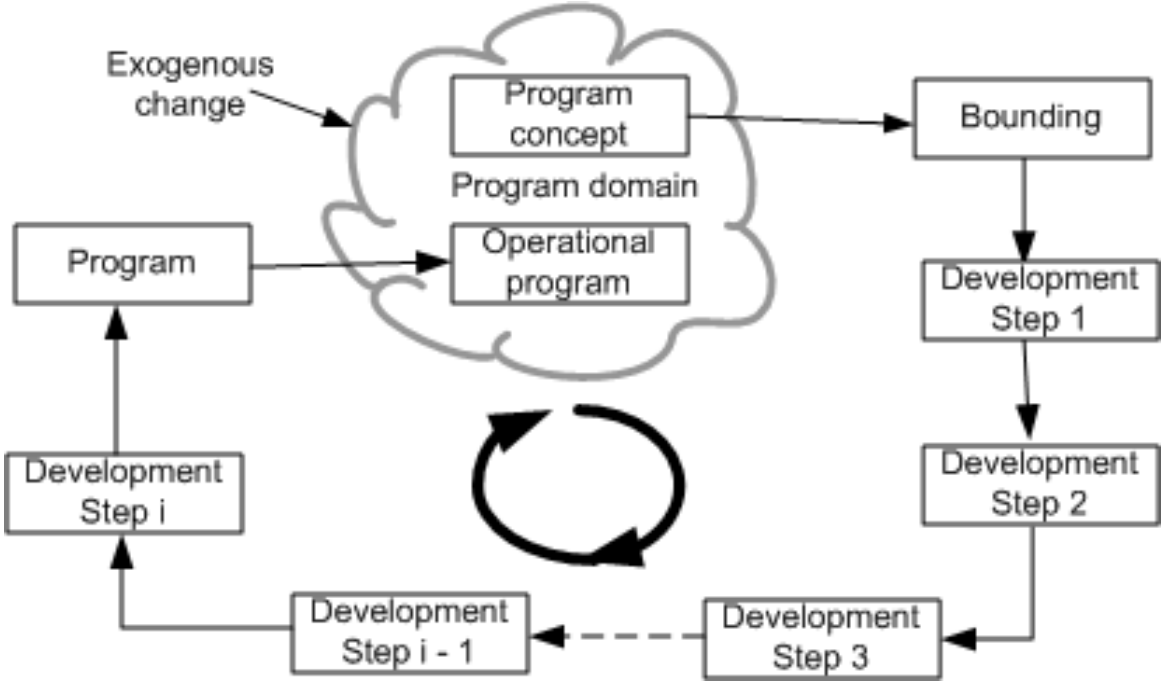


Figure 2.8 E-type programs with feedback © Wiley 2006

© Tripathy & Naik

2.3.2 Laws of Software Evolution

- Lehman formulated a set of observations that he called laws of evolution.
- The laws themselves have evolved from three in 1974 to eight by 1997.
- These laws are the results of studies of the evolution of large-scale proprietary or closed source system (CSS).
- The laws concern what Lehman called E-type systems:

> "Monolithic systems produced by a team within an organization that solve a real world problem and have human users."

- Lehman's laws were not meant to be used in a mathematical sense, as, say, Newton's laws are used in physics.
- The term "laws" was used because the observed phenomena were beyond the influence of managers and developers.
- The laws were an attempt to study the nature of software evolutions and the evolutionary trajectory likely taken by software.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

Names of the laws	Brief descriptions
I. Continuing change (1974)	E-type programs must be continually adapted, else they become progressively less satisfactory.
II. Increasing complexity (1974)	As an E-type program evolves, its complexity increases unless work is done to maintain or reduce it.
III. Self regulation (1974)	The evolution process of E-type programs is self regulating, with the time distribution of measures of processes and products being close to normal.
IV. Conservation of organizational stability (1978)	The average effective global activity rate in an evolving E-type program is invariant over the product's lifetime.
V. Conservation of familiarity (1978)	The average content of successive releases is constant during the life-cycle of an evolving E-type program.
VI. Continuing growth (1991)	To maintain user satisfaction with the program over its lifetime, the functional content of an E-type program must be continually increased.
VII. Declining quality (1996)	An E-type program is perceived by its stakeholders to have declining quality if it is not maintained and adapted to its environment.
VIII. Feedback system (1971-1996)	The evolution processes in E-type programs constitute multi-agent, multi-level, multi-loop feedback systems.

Table 2.5: Laws of software evolution, adapted from Lehman et al. [35] (©[1997] IEEE).

© Tripathy & Naik

2.3.2 Laws of Software Evolution

First Law Continuing change: E-type programs must be continually adapted, else they become progressively less satisfactory.

- Many assumptions are embedded in an E-type program.
- A subset of those assumptions may be complete and valid at the initial release of the product.
- As the application's environment changes in terms of the number of sophisticated users, a growing number of assumptions become invalid.
- Consequently, new requirements and new change requests will emerge.
- When the updated and modified program is reintroduced into the operational domain, it continues to satisfy user needs for a while.
- Next, more changes occur in the operation environment, additional user needs are identified, and additional change requests are made.
- As a result, the evolution process moves into a vicious cycle.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

Second Law Increasing complexity: As an E-type program evolves, its complexity increases unless work is done to maintain or reduce it.

- As the program evolves, its complexity grows because of the imposition of changes after changes on the program.
- In order to incorporate new changes, more objects, modules, and sub-systems are added to the system.
- These changes lead to a decline in the product quality, unless additional work is performed to arrest the decline.
- The only way to avoid this from happening is to invest in improving the structure of the software without adding to its functionality.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

Third Law Self regulation: The evolution process of E-type programs is self regulating, with the time distribution of measures of processes and products being close to normal.

- This law states that large programs have a dynamics of their own.
- Attributes such as size, time between releases, and the number of reported faults are approximately invariant from release to release.
- The various groups within the large organization apply constraining information controls and reinforcing information controls influenced by past and present performance indicators.
- Their actions control, check, and balance the resource usage, which is a kind of feedback-driven growth and stabilization mechanism.
- This establishes a self-controlled dynamic system whose process and product parameters are normally distributed as a result of a huge number of largely independent implementation and managerial decisions.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

Fourth Law Conservation of organizational stability: The average effective global activity rate in an evolving E-type program is invariant over the product's lifetime.

- This law suggests that most large software projects work in a "stationary" state, which means that changes in resources or staffing have small effects on long-term evolution of the software.

- To a certain extent management certainly do control resource allocation and planning of activities. However, as suggested by the third law, program evolution is essentially independent of management decisions..

- Activities during the lifecycle of a system are not exclusively decided by management, but by a wide spectrum of controls and feedback inputs.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

Fifth Law Conservation of familiarity: The average content of successive releases is constant during the life-cycle of an evolving E-type program.

The law suggests that one should not include a large number of features in a new release without taking into account the need for fixing the newly introduced faults.

Conservation of familiarity implies that maintenance engineers need to have the same high level of understanding of a new release even if more functionalities have been added to it.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

Sixth Law Continuing growth: To maintain user satisfaction with the program over its lifetime, the functional content of an E-type program must be continually increased.

- It is important to distinguish this law from the first law which focuses on "Continuing Change."
- The first law captures the fact that an E-type software's operational domain undergoes continual changes.
- Those changes are partly driven by installation and operation of the system and partly by other forces.
- An example of other forces is human desire for improvement and perfection.
- These two laws – the first and the sixth – reflect distinct phenomena and different mechanisms.
- When phenomena are observed, it is often difficult to determine which of the two laws underlies the observation.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

Seventh Law Declining quality: An E-type program is perceived by its stakeholders to have declining quality if it is not maintained and adapted to its environment.

- This law directly follows from the first and the sixth laws.
- An E-Type program must undergo changes in the forms of adaptations and extensions to remain satisfactory in a changing operational domain.
- Those changes are very likely to degrade the performance and will potentially inject more faults into the evolving program.
- In addition, the complexity of the program in terms of interactions between its components increases, and the program structure deteriorates.
- The term for this increase in complexity over time is called ***entropy***.
- The average rate at which software ***entropy*** increases is about 1-3 percent per calendar year.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

Seventh Law Declining quality: An E-type program is perceived by its stakeholders to have declining quality if it is not maintained and adapted to its environment.

- There is significant decline in stakeholder satisfaction because of growing entropy, declining performance, increasing number of faults, and mismatch of operational domains.
- The above factors also cause a decline in software quality from the user's perspective.
- The decline of software quality over time is related to the growth in entropy associated with software product **aging or code decay**.
- Therefore, it is important to continually undertake preventive measures to reduce the entropy by improving the software's overall architecture, high-level and low-level design, and coding.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

There are two types of aging in software lifecycles:

- **software process execution aging:** It manifests in degradation in performance or transient failures in continuously running the software system,
- **software product aging:** It manifests in degradation of quality of software code and documentation due to frequent changes.

The symptoms of Aging in software were:

- **Pollution:** Pollution means that there are many modules or components in a system which are not used in the delivery of the business functions of the system.
- **Embedded knowledge:** Embedded knowledge is the knowledge about the application domain that has been spread throughout the program such that the knowledge can not be precisely gathered from the documentation.
- **Poor lexicon:** Poor lexicon means that the component identifiers have little lexical meaning or are incompatible with the commonly understood meaning of the components that they identify.
- **Coupling:** Coupling means that the programs and their components are linked by an elaborate network of control flows and data flows.

2.3.2 Laws of Software Evolution

Code decay

The code is said to have decayed if it is very difficult to change it, as reflected by the following three key responses:

(i) the cost of the change, which is effective only on the personnel cost for the developers who implement it;
(ii) the calendar or clock time to make the changes; and
(iii) the quality of the changed software.

It is important to note that code decay is antithesis of evolution in the sense that while the evolution process is intended to make the code better, changes are generally degenerative thereby leading to code decay.

© Tripathy & Naik

2.3.2 Laws of Software Evolution

Eighth Law Feedback system: The evolution processes in E-type programs constitute multi-agent, multi-level, multi-loop feedback systems.

The eighth law is based on the observation that evolution process of the E-type software constitutes a multi-level, multi-loop, multi-agent feedback system:

(i) multi-loop means that it is an iterative process;

(ii) multi-level refers to the fact that it occurs in more than one aspect of the software and its documentation; and

(iii) a multi-agent software system is a computational system where software agents cooperate and compete to achieve some individual or collective tasks. Feedback will determine and constrain the manner in which the software agents communicate among themselves to change their behavior.

© Tripathy & Naik

2.3.3 Empirical Studies

- In circa 1976, Belady and Lehman studied 20 releases of the OS/360 O.S.
- The results of their study led them to postulate five laws of software evolution:
  Continuing change,
  Increasing complexity,
  Self regulation,
  Conservation of organizational stability, and
  Conservation of familiarity.
- Yuen, a collaborator of Lehman, notes that the last three of the aforementioned laws are more based upon those of human organizations involved in the maintenance process rather than the properties of the software itself.

© Tripathy & Naik

2.3.3 Empirical Studies

- In a project entitled FEAST (Feedback, Evolution, And Software Technology), Lehman and his colleagues studied evolution of releases from four CSS systems:
  (i) two operating systems (OS/360 of IBM and VME OS of ICL),
  (ii) one financial system (Logica's FW banking transaction system), and
  (iii) a real-time telecommunication system (Lucent Technologies).
- The studies suggest that during the maintenance process a system tracks a growth curve that can be approximated either as linear or inverse-square.
- The inverse-square model represents the growth phenomena as an inverse square of the continuing effort.
- Those trends increase the confidence of validity of the following six laws: Continuing change (I), Increasing complexity (II), Self regulation (III), Conservation of familiarity (V), Continuing growth (VI), Feedback system (VII).
- Confidence in the seventh law "Declining quality" is based on the theoretical analysis, whereas the fourth law "Conservation of organizational stability" is neither supported nor falsified based on the metric presented.

2.3.5 Evolution of FOSS Systems

- The FOSS movement is attributed to Richard M. Stallman, who started the GNU (O.S) project.
- FOSS – also referred to as FLOSS (Free/Libre/Open Sources Software).
- FOSS is a class of software that is both free software and open source.
- FOSS have lots of new characteristics. Eric Raymond concisely documented the FOSS approach in an article entitled "The Cathedral and the Bazaar."

FOSS is made available to the general public with either relaxed or non-existent intellectual property restrictions. The free emphasizes the freedom to modify and redistribute under the terms of the original license while open emphasizes the accessibility to the source code



Richard M. Stallman

© Tripathy & Naik

2.3.5 Evolution of FOSS Systems

- There are huge differences between the evolutions of FOSS based software and CSS based software in terms of:
  (i) team structure,
  (ii) process,
  (iii) releases, and
  (iv) global factors.
- Example FOSS:

Linux: Started by Linus Benedict Torval.



Linus benedict Torval

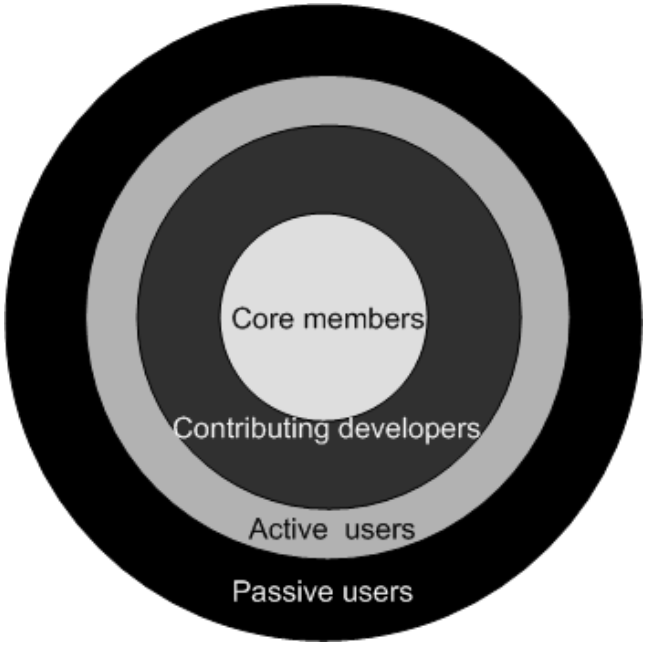


Figure 2.9 Onion model of FOSS development structure

© Tripathy & Naik

2.3.5 Evolution of FOSS Systems

- In circa 1988, Pirzada pointed out the differences between the evolution of the Unix OS and system studied by Lehman.
- Pirzada argued that the differences in academic and industrial s/w development could lead to a differences in the evolutionary pattern.
- In circa 2000, empirical study of free and open source software (FOSS) evolution was conducted by Godfrey and Tu.
- They found that the growth trends from 1994-1999 for the evolution of FOSS Lunix OS to be super-linear, that is greater than linear.
- Robles and his collaborator concluded that Lehman's laws, 3, 4, and 5 are not fitted to large scale FOSS system such as Linux.

© Tripathy & Naik

2.3.5 Evolution of FOSS Systems

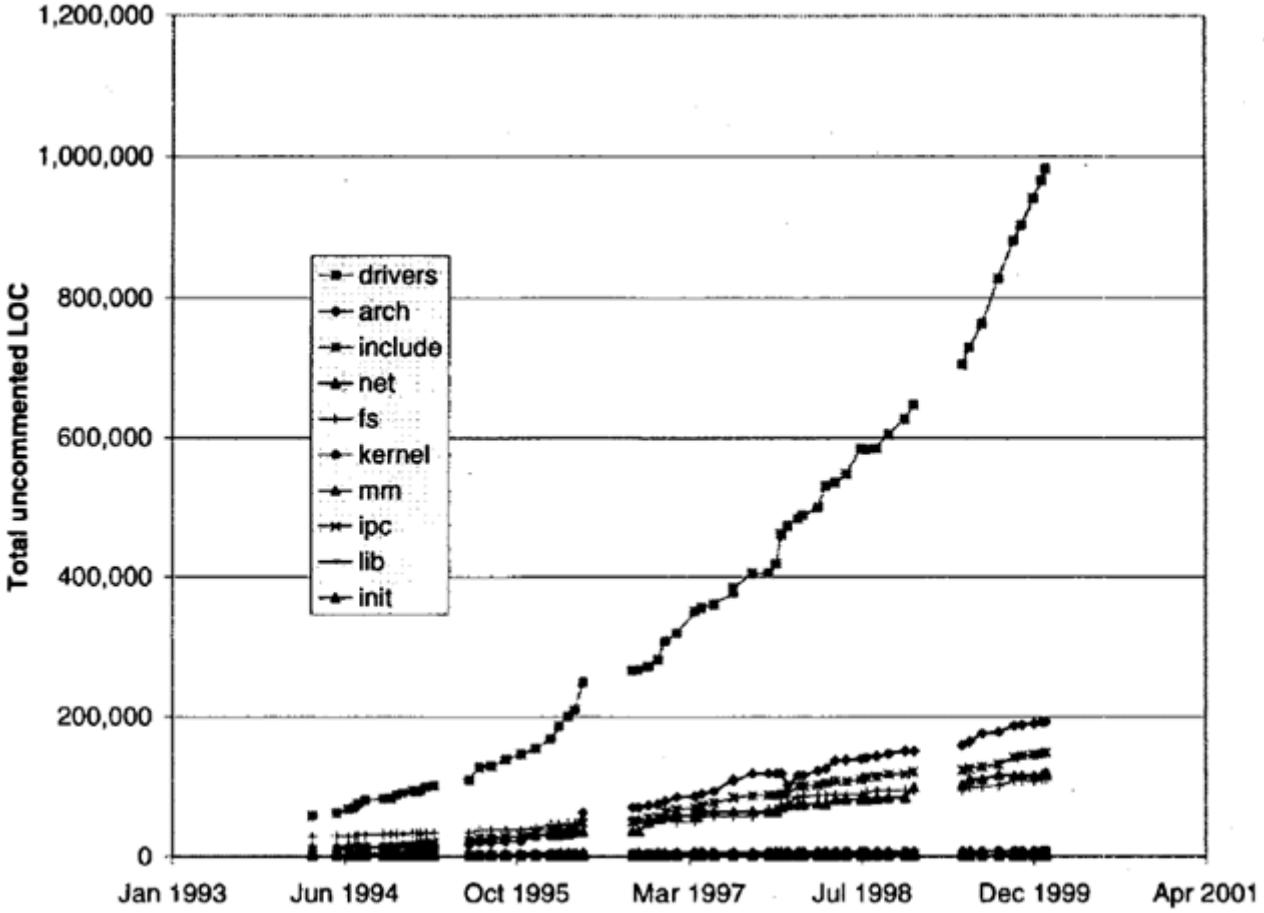


Figure 2.10: Growth of the major subsystems (development releases only) of the Linux OS

© Tripathy & Naik

2.4 Maintenance of COTS-based Systems

- A commercial off-the-shelf (COTS) component is defined as:

A unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third parties.

- The use of COTS components is increasing in modern software development because of the following reasons:

(i) there is significant gain in productivity due to reusing commercial components; (ii) the time to market the product decreases;

(iii) The product quality is expected to be high, assuming that the COTS components have been well tested; and

(iv) there is efficient use of human resources due to the fact that development personnel will be freed up for other tasks.

- The black-box nature of COTS components prevents system integrators from modifying the components to better meet user's requirements.
- The integrators have no visibility of the details of the components, their evolutions, or their maintenance.

© Tripathy & Naik

2.4 Maintenance of COTS-based Systems

- The only source code being written and modified by the integrators is what is needed for integrating the COTS based systems.
- This includes code for:
  (i) Tailoring and Wrapping the individual components, and
  (ii) Glue code required to put the components together.

Wrapper: It is a piece of code that one builds to isolate the underlying components from other components of the system.

Glue: A glue component provides the functionality to combine different components.

Tailoring: Components tailoring refers to the ability to enhance the functionality of a component.

Tailoring is done by adding some elements to a component to enrich it with a functionality not provided by the vendor.

Tailoring does not involve modifying the source code of the component.

© Tripathy & Naik

2.4.1 Why Maintenance of CBS is Difficult?

- Frozen functionality.
- Incompatibility of upgrades.
- Trojan horses.
- Unreliable COTS components.
- Defective middleware.

© Tripathy & Naik

2.5 Summary

- General Idea
- Categories of Maintenance Concepts
- Evolution of Software Systems
- Maintenance of COTS-based Systems

© Tripathy & Naik